

Parallel Computing Architectures

Pacheco, Chapter 2

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

<https://www.moreno.marzolla.name/>

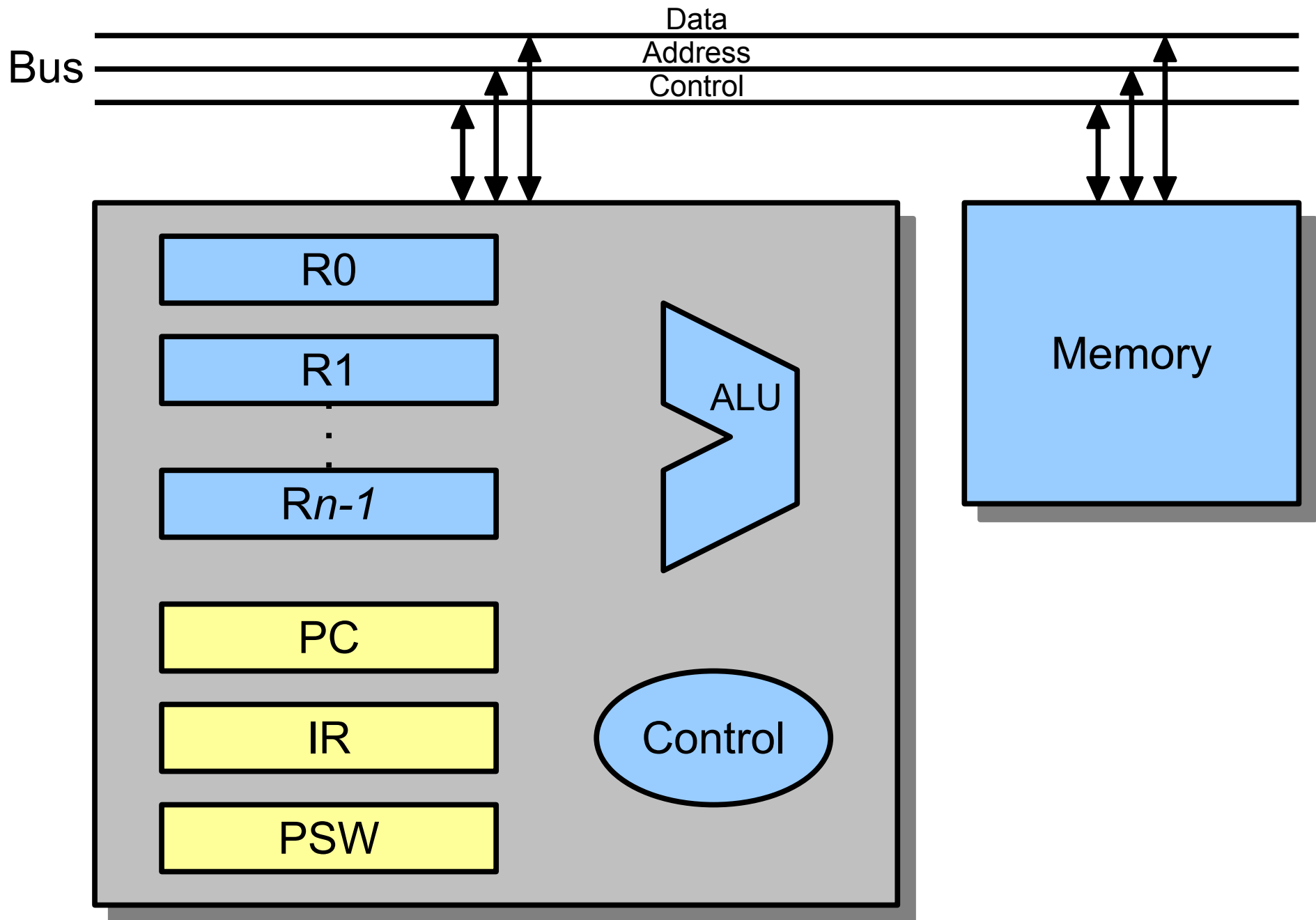
Copyright © 2013—2020, 2024, 2025
Moreno Marzolla, Università di Bologna, Italy
<https://www.moreno.marzolla.name/teaching/APAI/>



This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

Computer Architectures (recap)

Von Neumann architecture



- **R_0 – R_{n-1} (General Purpose Registers):** Small, fast storage locations (8-bit in this example) used for operands, intermediate results, temporary values, and sometimes addresses. Their use reduces frequent access to main memory.
- **PC (Program Counter):** Holds the memory address of the next instruction to be fetched. Automatically increments after each fetch unless modified by a control transfer (branch, jump, interrupt, subroutine call).
- **IR (Instruction Register):** Stores the currently fetched instruction. Its contents are decoded by the control unit to generate the necessary signals for execution.
- **PSW (Program Status Word):** A special register containing condition codes and control flags, such as Zero, Carry, Overflow, Sign, Interrupt Enable/Disable, and mode bits (user/supervisor). It reflects the current execution state of the processor.
- **Control Unit:** Directs the flow of data and operations. It interprets the instruction in the IR, orchestrates ALU operations, manages register transfers, and issues memory read/write and I/O commands.

Other key components in the Von Neumann model:

- **ALU (Arithmetic Logic Unit):** Executes arithmetic (addition, subtraction, etc.) and logical operations (AND, OR, NOT, comparisons) on data from registers or memory.
- **Memory Unit:** Unified storage for both program instructions and data. Accessed over a shared bus using addresses provided by the CPU.
- **Bus System:** A common pathway (data bus, address bus, control bus) interconnecting CPU, memory, and I/O. In Von Neumann, both instructions and data share the same bus, leading to the "Von Neumann bottleneck."

CPU times compared to the real world

	CPU	Real world
1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access	120 ns	6 min
Solid-state disk I/O	50-150 μ s	2-6 days
Rotational disk I/O	1-10 ms	1-12 months
Internet: SF to NYC	40 ms	4 years
Internet: SF to UK	81 ms	8 years
Internet: SF to Australia	183 ms	19 years
Physical system reboot	1 min	6 millennia

Processor/Memory wall

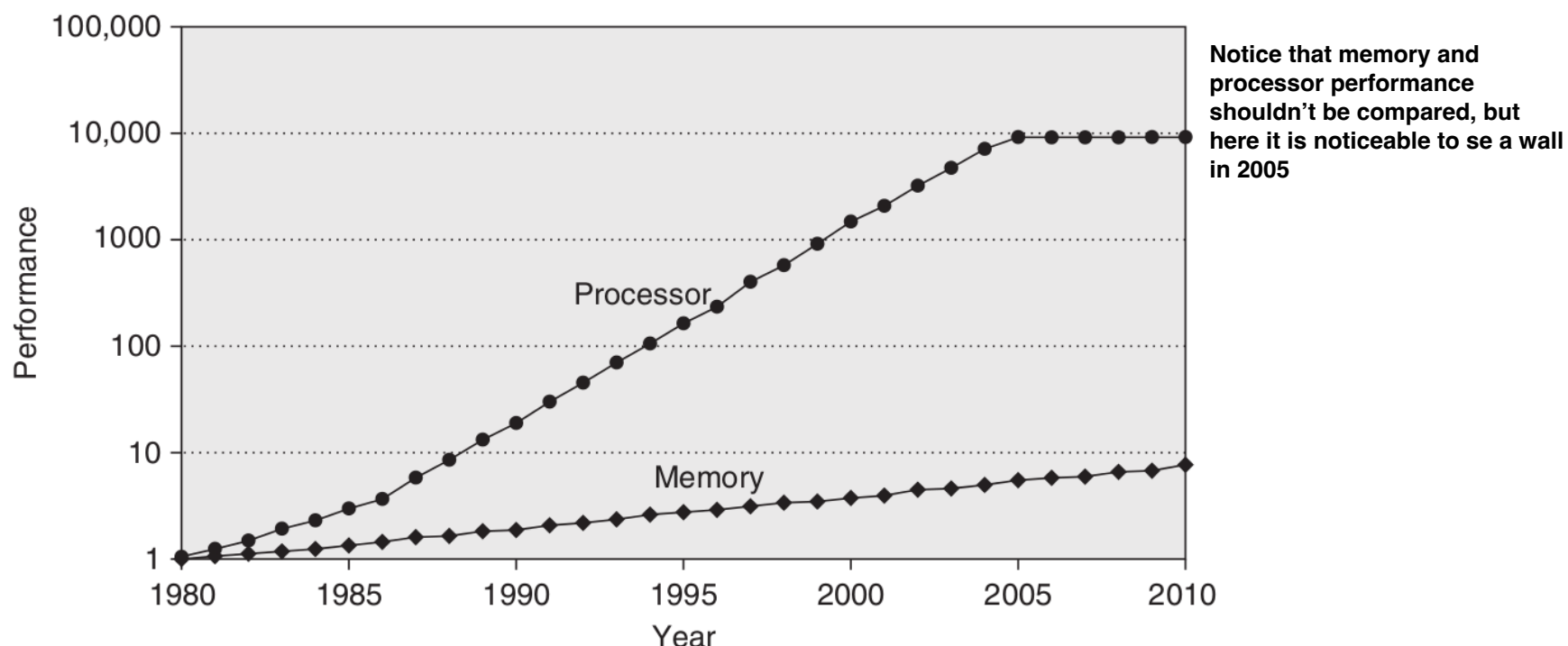
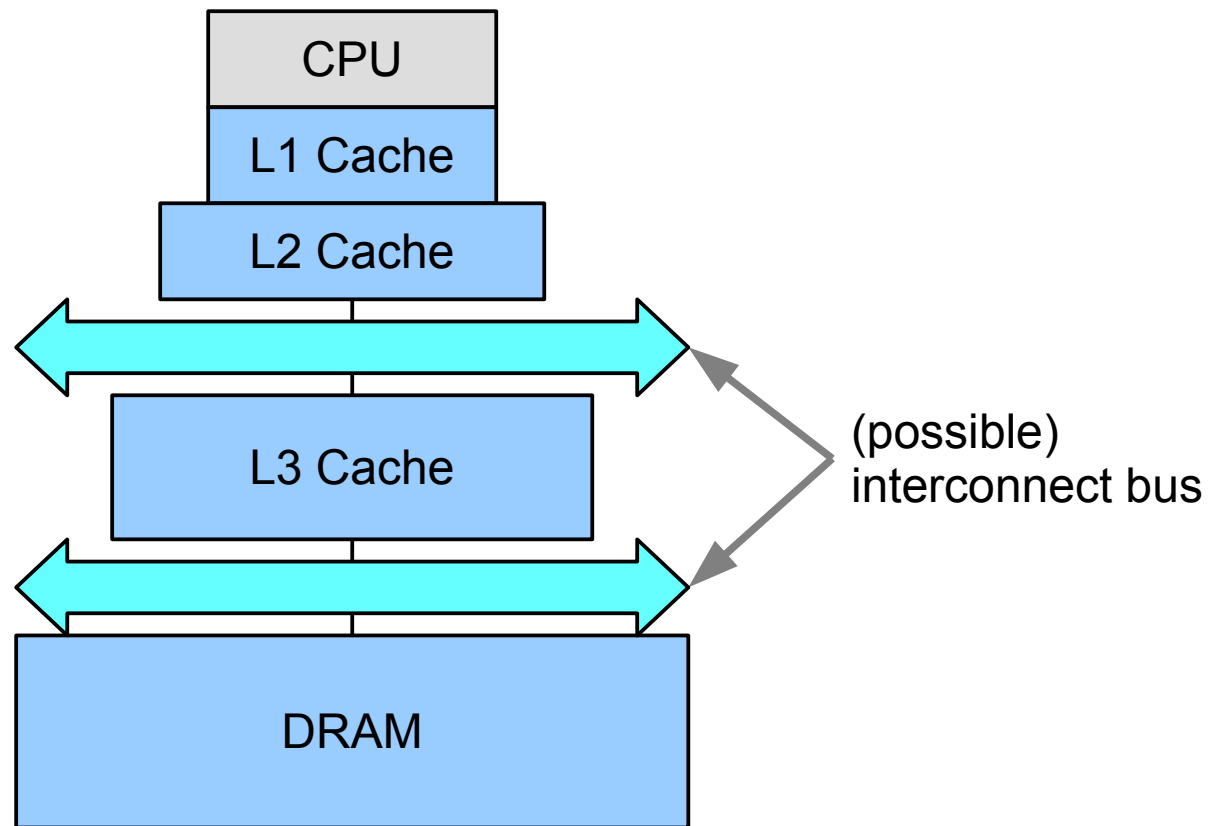


Figure 2.2 Starting with 1980 performance as a baseline, the gap in performance, measured as the difference in the time between processor memory requests (for a single processor or core) and the latency of a DRAM access, is plotted over time. Note that the vertical axis must be on a logarithmic scale to record the size of the processor-DRAM performance gap. The memory baseline is 64 KB DRAM in 1980, with a 1.07 per year performance improvement in latency (see [Figure 2.13](#) on page 99). The processor line assumes a 1.25 improvement per year until 1986, a 1.52 improvement until 2000, a 1.20 improvement between 2000 and 2005, and no change in processor performance (on a per-core basis) between 2005 and 2010; see [Figure 1.1](#) in [Chapter 1](#).

Addressing Von Neumann bottlenecks: Caching

Cache hierarchy

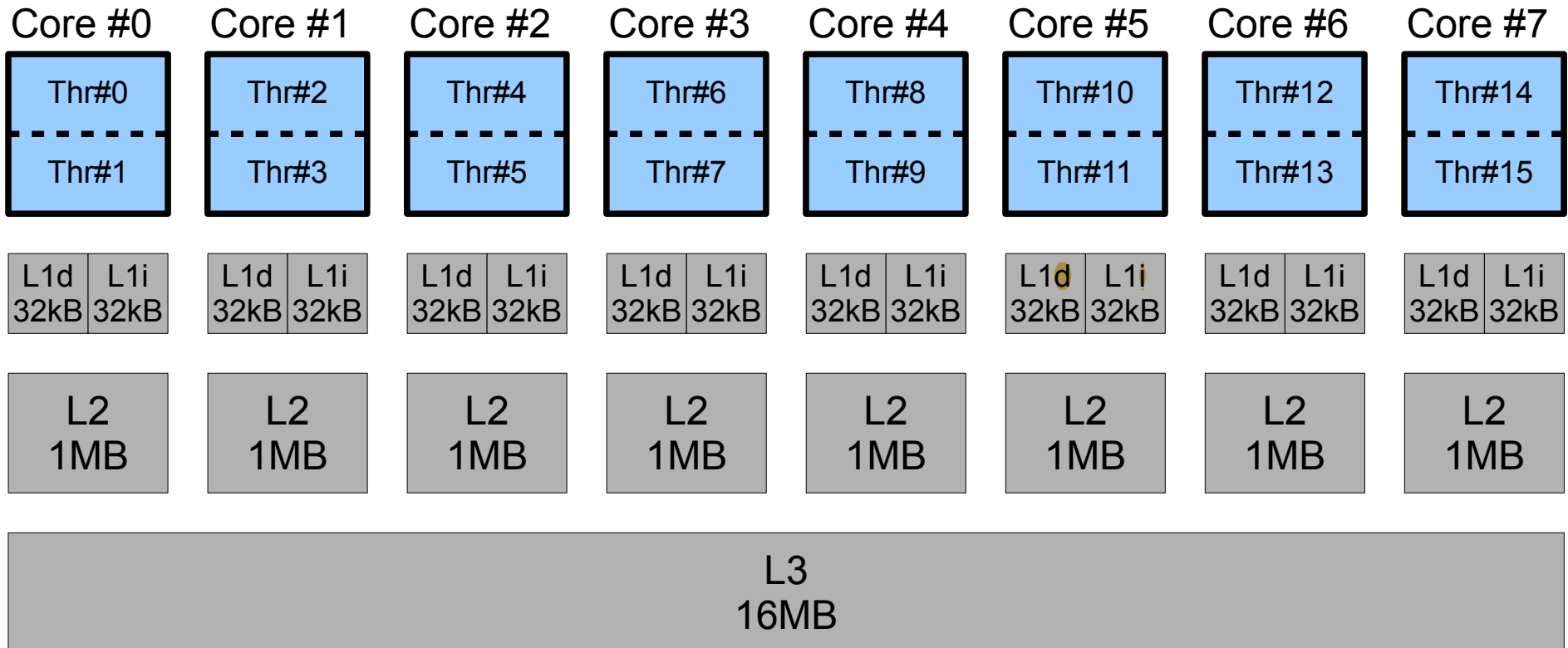
- Large memories are slow; fast memories are small



Example of cache hierarchy

L1d - data partition of level 1 cache

L1i - instructions partition of level 1 cache



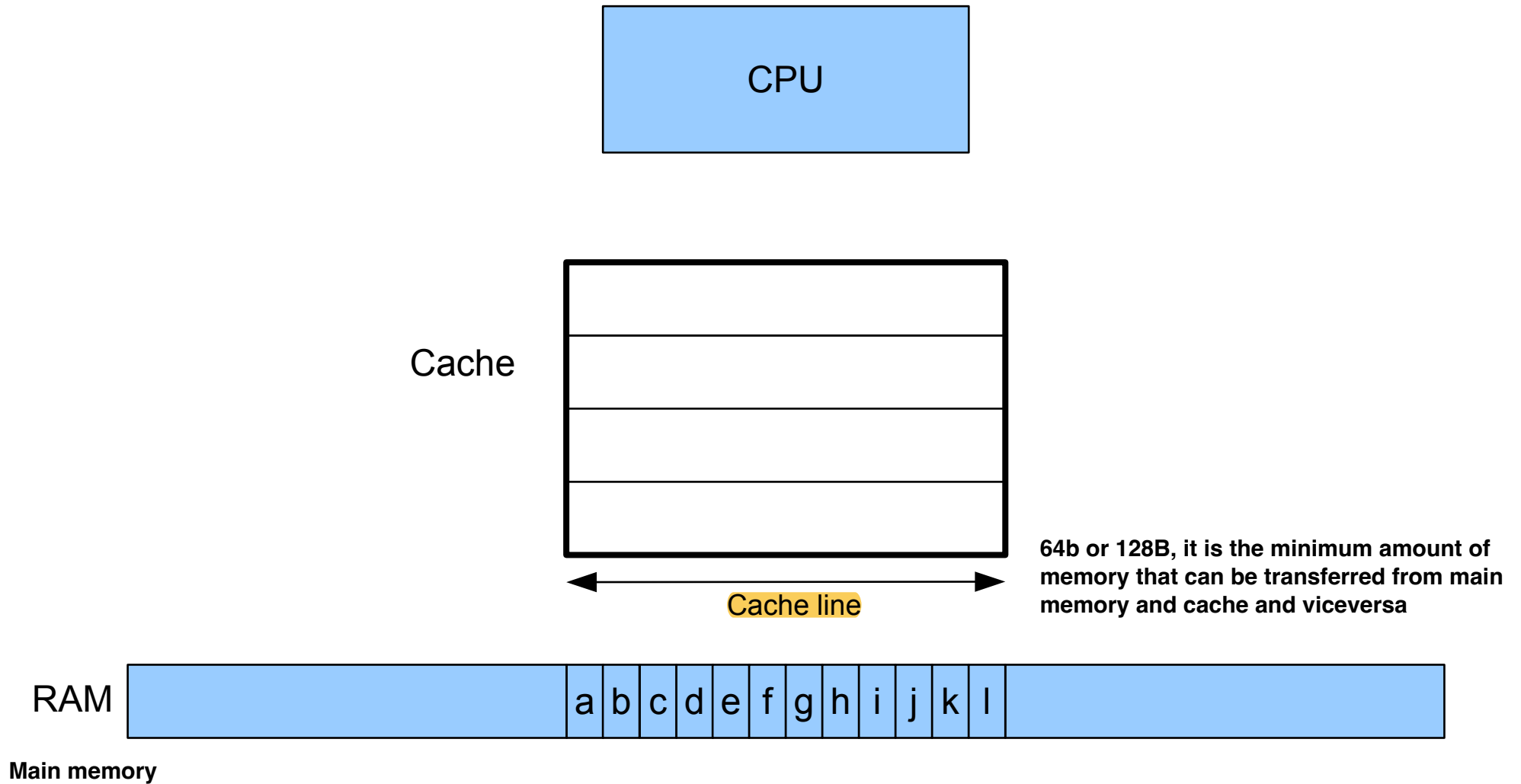
How caches work

- Cache memory is very fast
 - Often located inside the processor chip
 - Expensive, small compared to system memory
- The cache contains a copy of the content of some recently accessed (main) memory locations
- If the same memory location is accessed again, and its content is in cache, then the cache is accessed instead of the system RAM

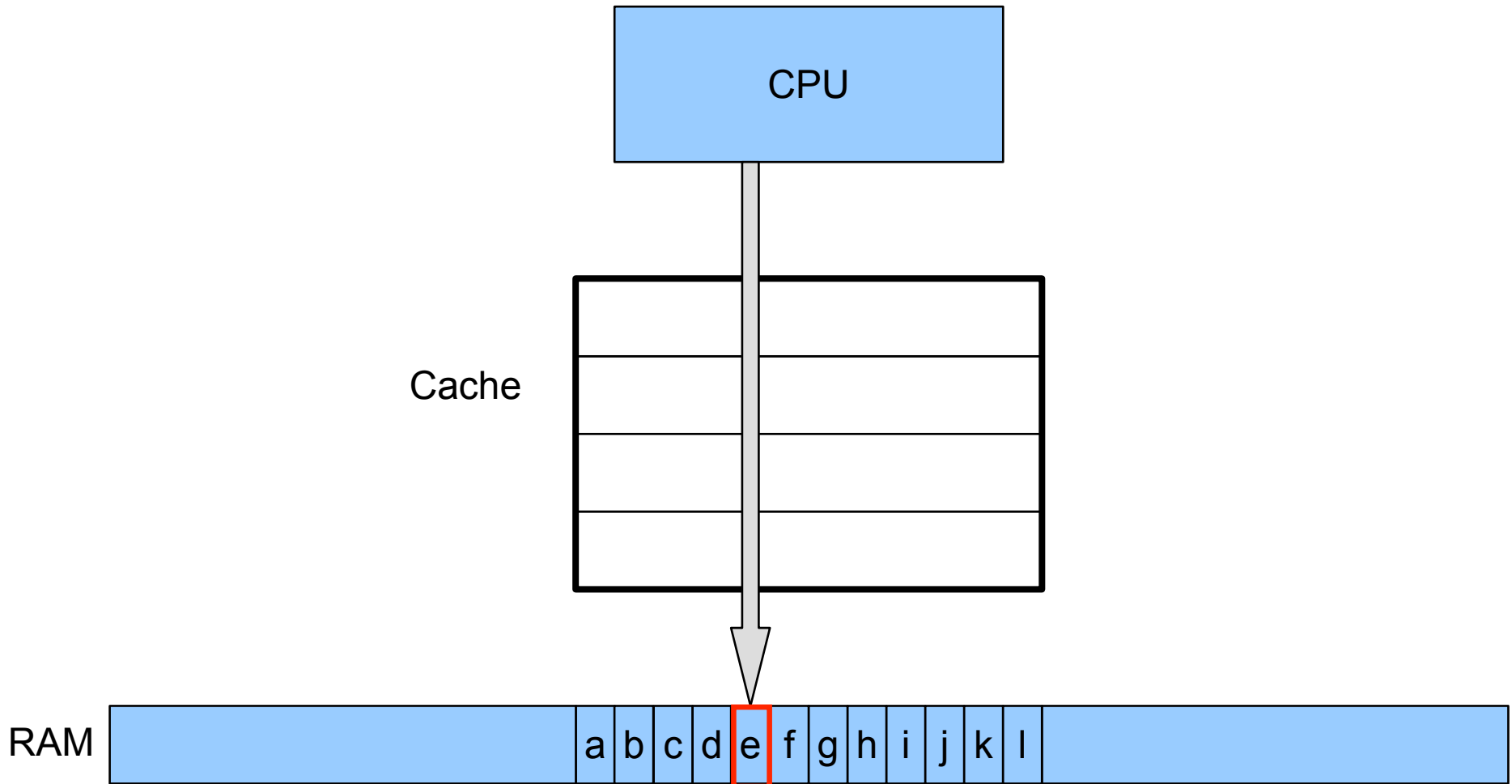
How caches work

- If the CPU accesses a memory location whose content is not already in cache
 - the content of that memory location and "some" adjacent locations are copied in cache
 - in doing so it might be necessary to purge some old data from the cache to make room to the new data
- The smallest unit of data that can be transferred to/from the cache is the **cache line**
 - On most processors a cache line is 64B or 128B

Example

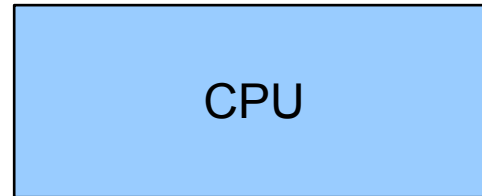


Example

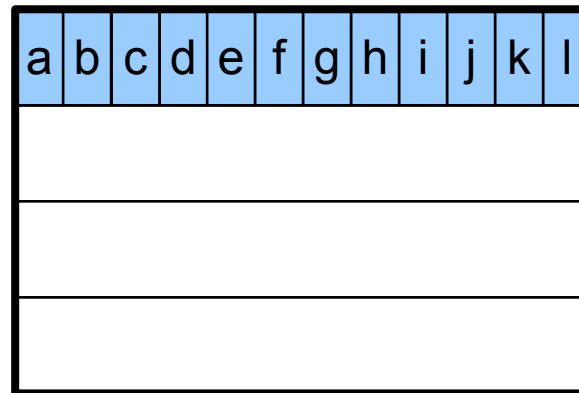


The only requested byte but it stores all the cache line

Example

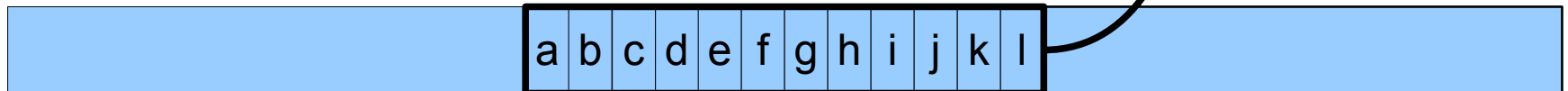


Cache

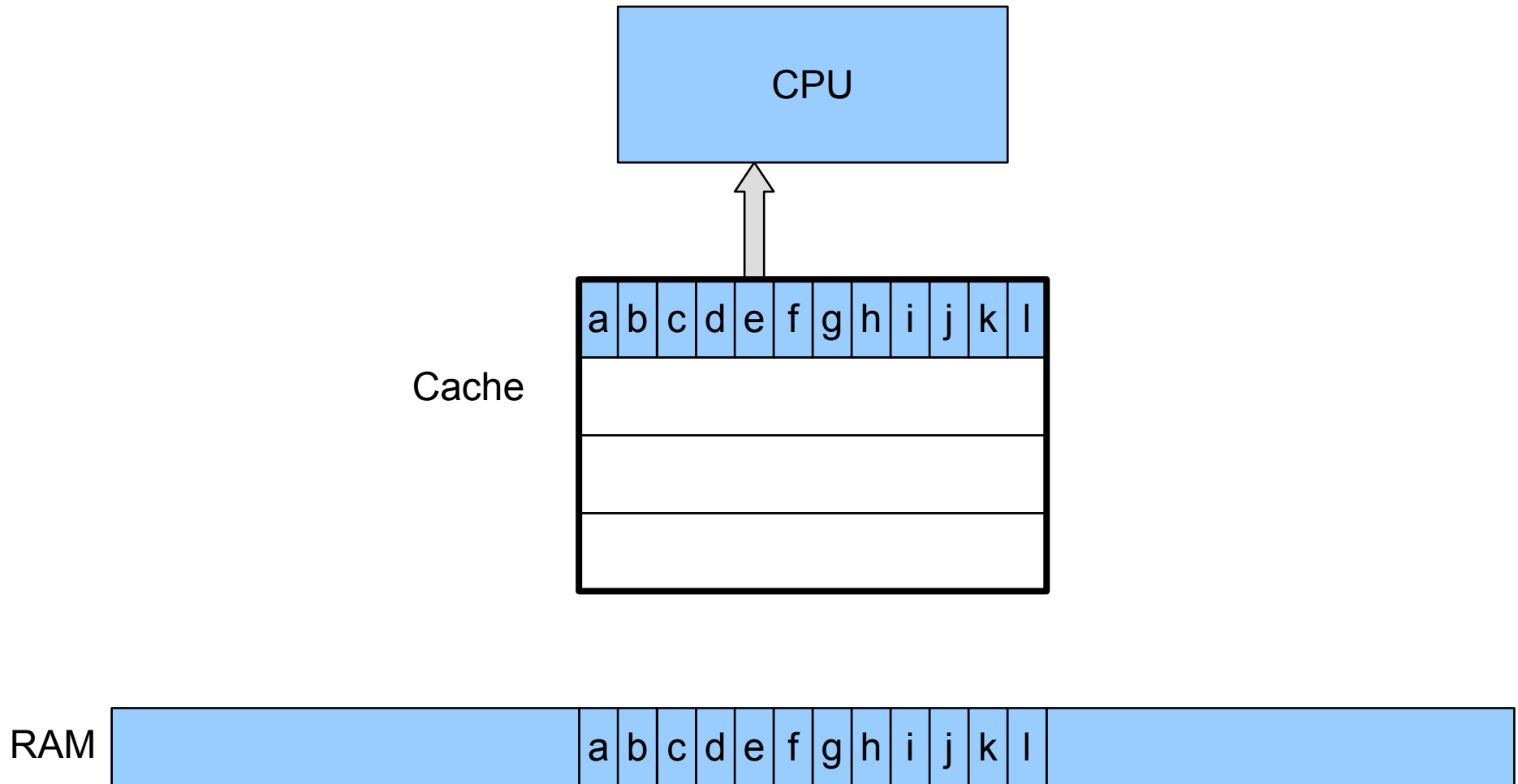


This procedure is less efficient than storing only that exact byte in principle but most of the applications show the locality of access (both spatial and temporal), making nearby bytes very likely to be requested in following cycles

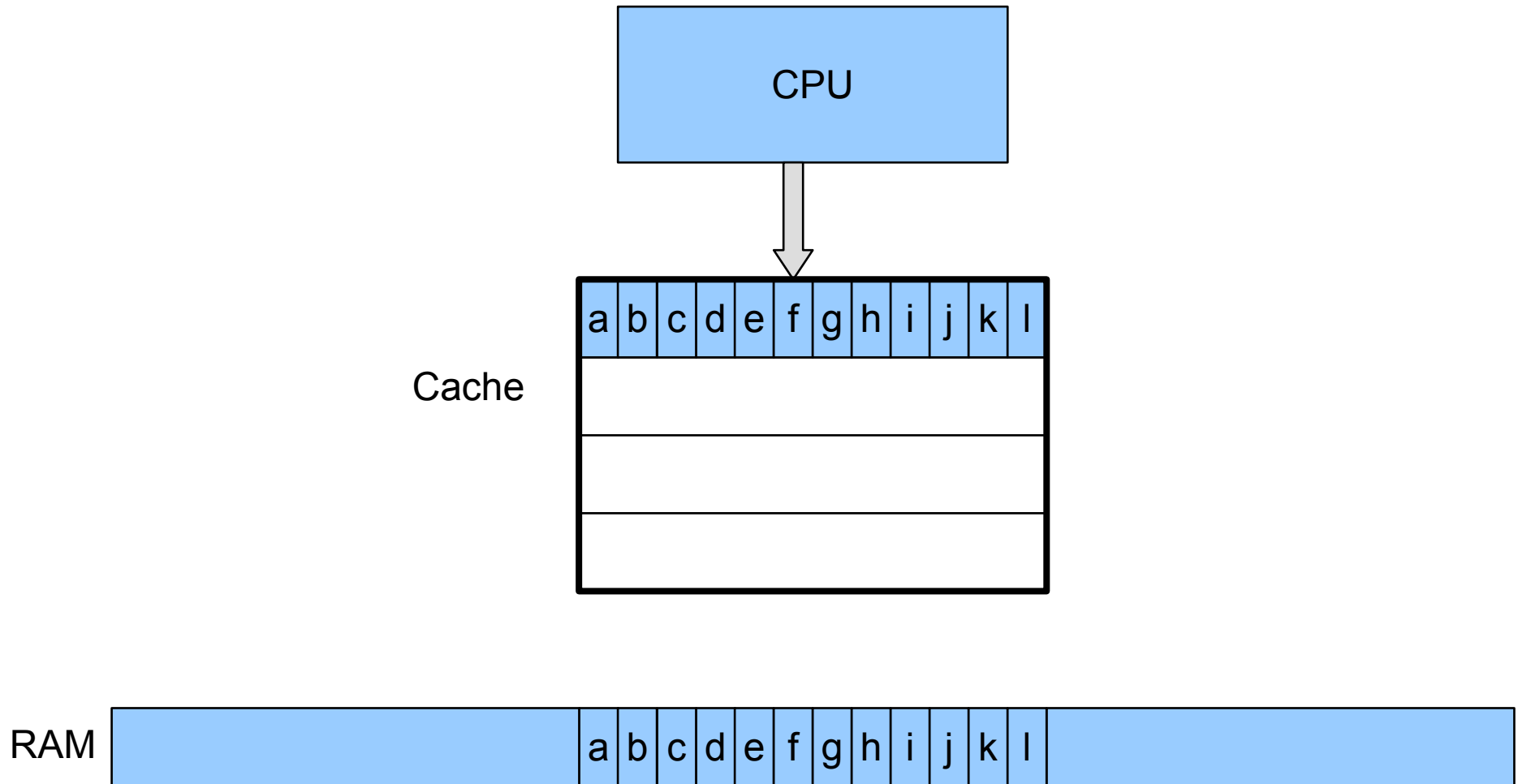
RAM



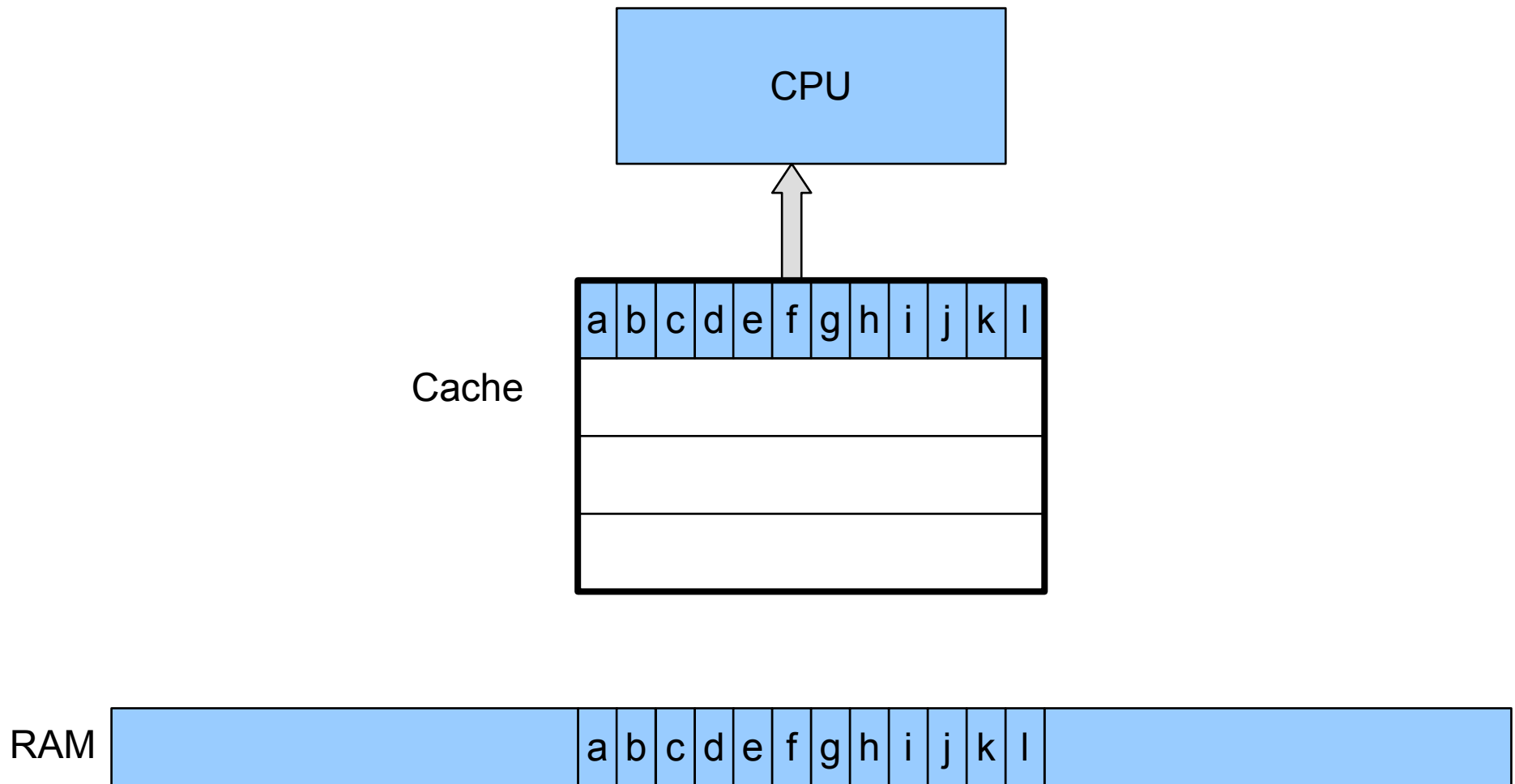
Example



Example



Example

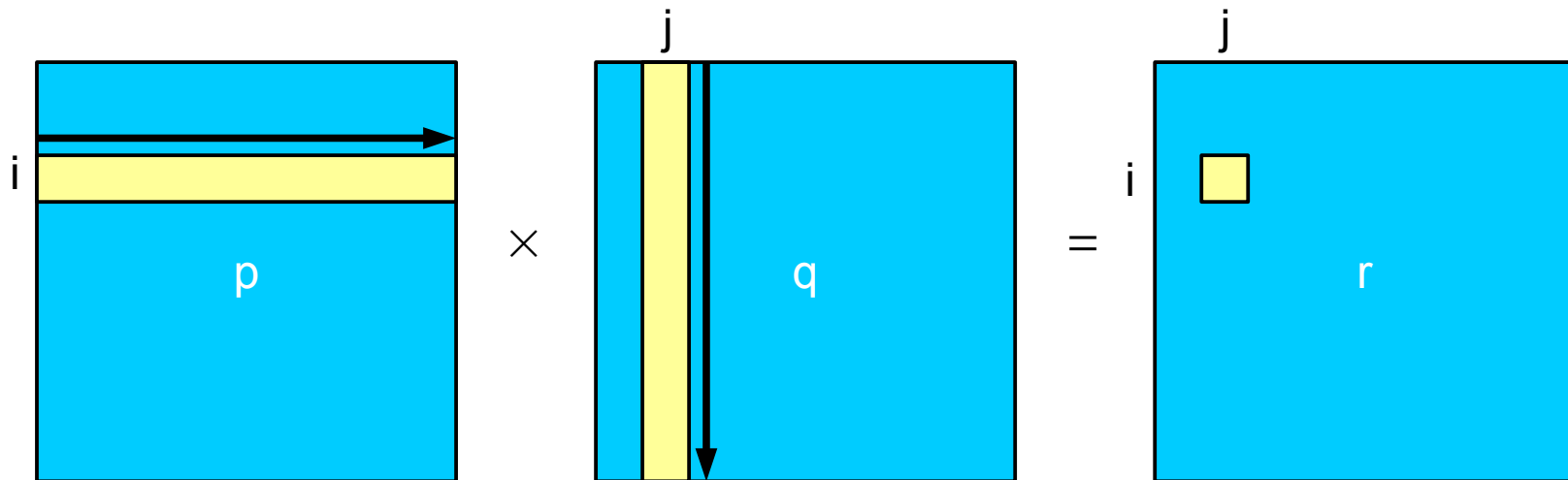


Spatial and temporal locality

- Cache memory works well when applications exhibit **spatial** and/or **temporal** locality
- **Spatial locality**
 - Accessing adjacent memory locations is OK
- **Temporal locality**
 - Repeatedly accessing the same memory location(s) is OK

Example: matrix-matrix product

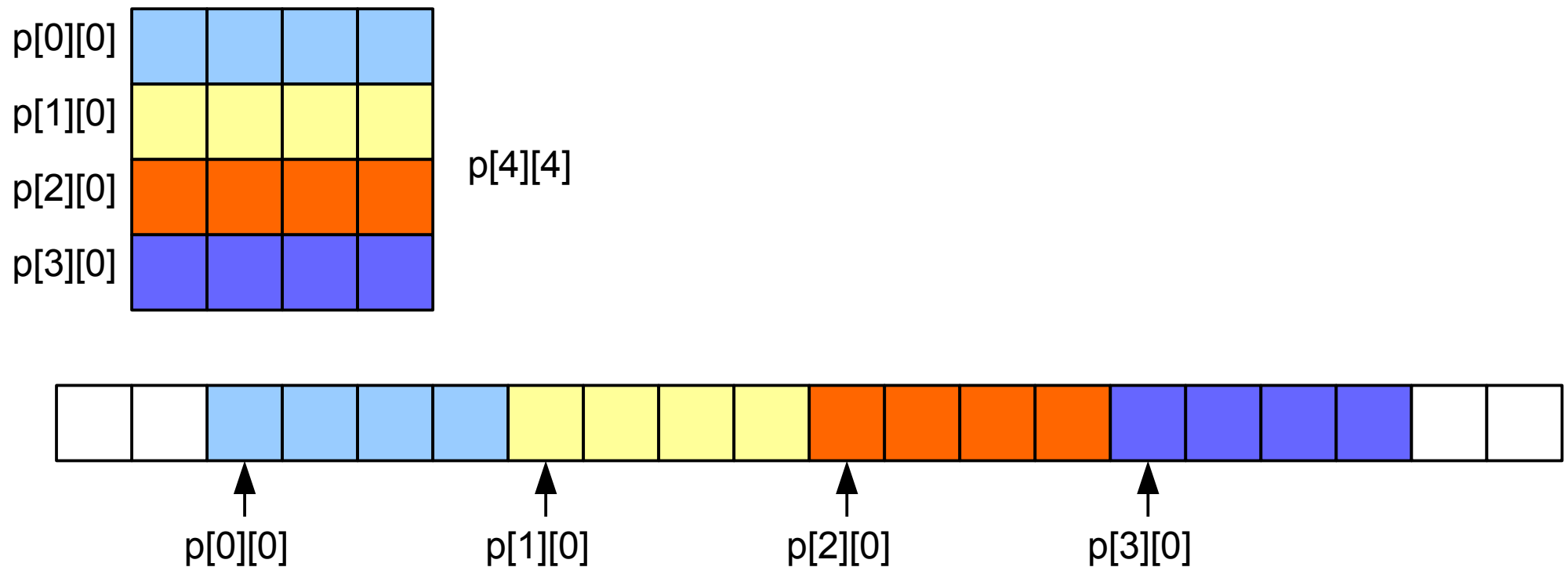
- Given two square matrices p , q , compute $r = p \times q$



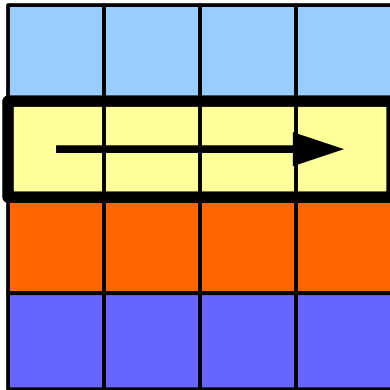
```
void matmul( double *p, double* q, double *r, int n)
{
    int i, j, k;
    for (i=0; i<n; i++) {
        for (j=0; j<n; j++) {
            double s = 0.0;
            for (k=0; k<n; k++) { s += p[i*n + k] * q[k*n + j]; }
            r[i*n + j] = s;
        }
    }
}
```

Matrix representation

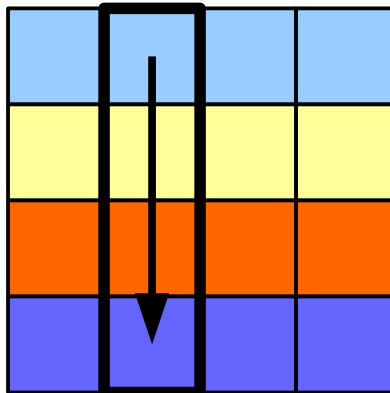
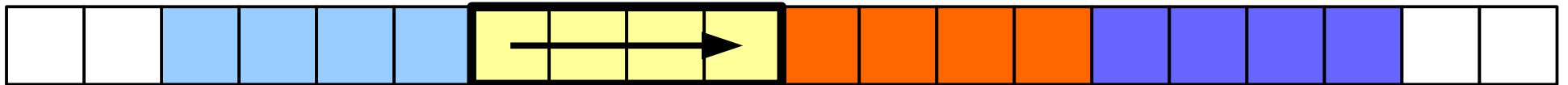
- Matrices in C are stored in *row-major order*
 - Elements of each row are contiguous in memory
 - Adjacent rows are contiguous in memory



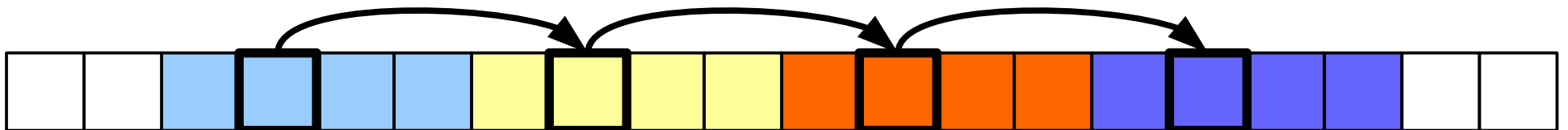
Row-wise vs column-wise access



Row-wise access is OK: the data is contiguous in memory, so the cache helps (spatial locality)

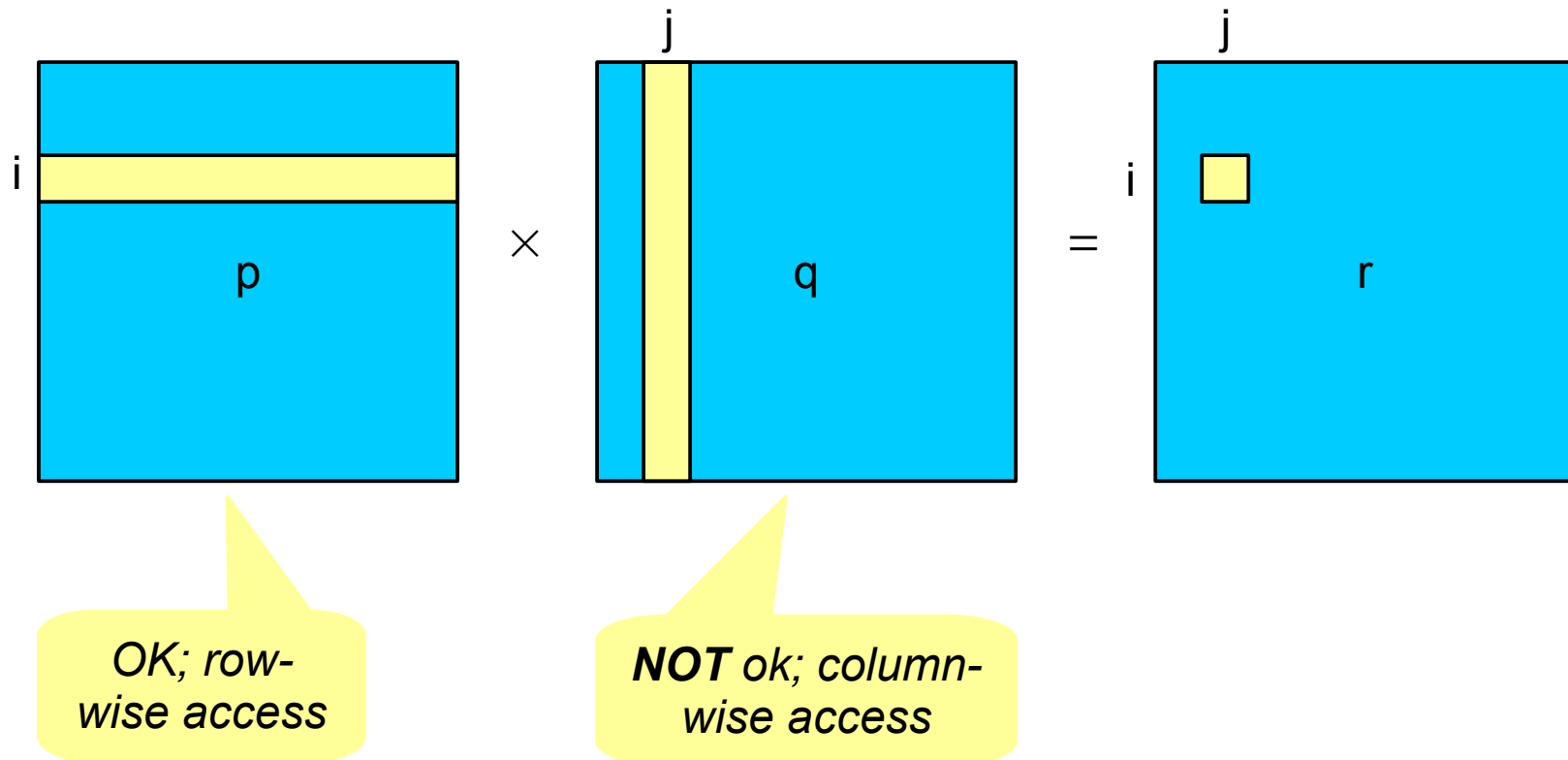


Column-wise access is **NOT OK**: the accessed elements are not contiguous in memory (*strided* access) so the cache does NOT help



Matrix-matrix product

- Given two squared matrices p , q , compute $r = p \times q$



Optimizing the memory access pattern

p00	p01	p02	p03
p10	p11	p12	p13
p20	p21	p22	p23
p30	p31	p32	p33

 $\times$

q00	q01	q02	q03
q10	q11	q12	q13
q20	q21	q22	q23
q30	q31	q32	q33

 $=$

r00	r01	r02	r03
r10	r11	r12	r13
r20	r21	r22	r23
r30	r31	r32	r33

p q r

 Transpose q

We need to allocate the memory for q^T

p00	p01	p02	p03
p10	p11	p12	p13
p20	p21	p22	p23
p30	p31	p32	p33

 $\times$

q00	q10	q20	q30
q01	q11	q21	q31
q02	q12	q22	q32
q03	q13	q23	q33

 $=$

r00	r01	r02	r03
r10	r11	r12	r13
r20	r21	r22	r23
r30	r31	r32	r33

p q^T r

We have to copy the matrix, then transposing the copy and finally computing the row-wise product of the 2 matrices. It comes up that the total time of this operation is still much more convenient than column-wise multiplication (9s vs 6s with 1024x1024 matrix).

But...

- Transposing q requires time. Is it worth it?
 - See [matmul.c](#)
 - `make matmul` produces two executables: `matmul-plain` and `matmul-transpose`
 - To measure the cache performance we use [perf](#)
 - On Debian/Ubuntu:

```
sudo apt-get install linux-tools-common linux-tools-generic
```
 - Which counters are available is system-dependent 😞
 - The meaning of the counters is system-dependent 😞😞😞
- THEY ARE HARDWARE COUNTERS ==> NO OVERHEAD

Number of memory load operation that didn't find the data in L1, so they then explored other caches

Measuring cache stats

```
perf stat -B -e \  
    task-clock,cycles,cache-references,\  
    cache-misses,L1-dcache-loads,L1-dcache-load-misses,\  
    L1-dcache-stores,L1-dcache-store-misses \  
    ./exec_name
```

```
$ ./cache-stats.sh ./matmul-plain
```

```
Starting plain matrix-matrix multiply (n=1000)... done
```

```
r[0][0] = 243.361596
```

```
elapsed time = 3.894047 s
```

*Low level cache
references and misses*

```
Performance counter stats for './matmul-plain':
```

3909,022101	task-clock (msec)	#	0,997	CPUs utilized
13.544.312.192	cycles	#	3,465	GHz
126.008.898	cache-references	#	32,235	M/sec
33.267.819	cache-misses	#	26,401	% of all cache refs
14.061.431.913	L1-dcache-loads	#	3597,174	M/sec
1.129.759.631	L1-dcache-load-misses	#	8,03%	of all L1-dcache hits
0	L1-dcache-stores	#	0,000	K/sec
1.776.808	L1-dcache-store-misses	#	0,455	M/sec

```
3,919924574 seconds time elapsed
```

*L1 data cache references
and misses*

As we can see here we decreased of one order of magnitude the amount of calls to other caches, showing why we are faster with row wise matrix multiplication.

Measuring cache stats

```
perf stat -B -e \  
    task-clock,cycles,cache-references,\  
    cache-misses,L1-dcache-loads,L1-dcache-load-misses,\  
    L1-dcache-stores,L1-dcache-store-misses \  
    ./exec_name
```

```
$ ./cache-stats.sh ./matmul-transpose
```

```
Starting transpose matrix-matrix multiply (n=1000)... done
```

```
r[0][0] = 243.361596
```

```
elapsed time = 2.656615 s
```

*Low level cache
references and misses*

```
Performance counter stats for './matmul-transpose'
```

2678,964056	task-clock (msec)	#	0,999	CPUs utilized
9.271.087.662	cycles	#	3,461	GHz
3.305.170	cache-references	#	1,234	M/sec
1.292.062	cache-misses	#	39,092	% of all cache refs
14.073.347.851	L1-dcache-loads	#	5253,280	M/sec
127.489.346	L1-dcache-load-misses	#	0,91%	of all L1-dcache hits
0	L1-dcache-stores	#	0,000	K/sec
1.960.718	L1-dcache-store-misses	#	0,732	M/sec

```
2,682123014 seconds time elapsed
```

*L1 data cache references
and misses*

Hardware Multithreading

In order to address the limitation of the Von Neumann architecture of one operation per time:

1) PIPELINE:

Split instruction execution into several stages (fetch, decode, execute, memory, writeback). Each stage is handled by a different functional unit, so in one cycle multiple instructions can be in different stages. Throughput approaches one instruction per cycle.

2) MULTIPLE ISSUE:

Allowing the processor to start several instructions in the same clock cycle instead of just one. There are two variants:

a) Static multiple issue (VLIW – Very Long Instruction Word):

The compiler groups independent instructions into a wide instruction word. At runtime the hardware just executes what the compiler scheduled. Hardware is simple, but compiler must do heavy analysis. Problem: binary compatibility is hard because instruction scheduling depends on the exact hardware.

b) Dynamic multiple issue (Superscalar):

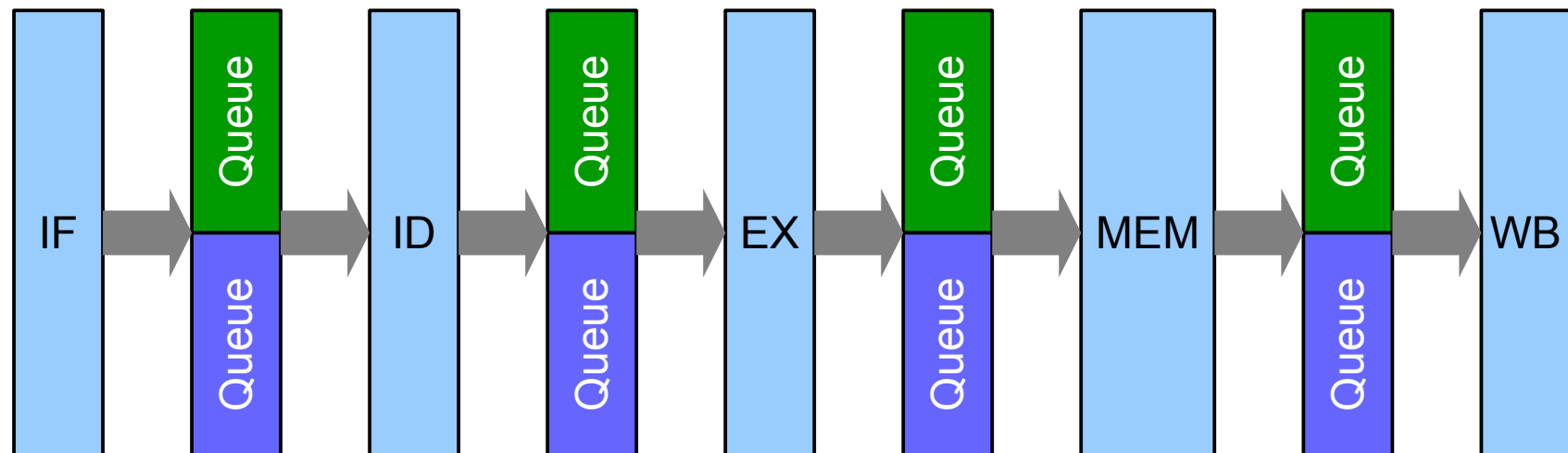
The processor fetches a normal sequence of instructions and the hardware itself decides, at each cycle, how many independent instructions it can launch in parallel. Requires hardware mechanisms: instruction window, dependency checking, register renaming, out-of-order execution, branch prediction. More complex hardware, but easier for the programmer and more portable.

Hardware Multithreading

- Improves the utilization of hardware resources in conventional, sequential processors
 - Both Intel and AMD have their own implementation of Hardware Multithreading
- A physical core is seen by the OS as **two** virtual processors
- Each virtual processor maintains a complete set of the architecture state:
 - general-purpose registers
 - control registers
 - advanced programmable interrupt controller (APIC) registers
 - some machine state registers
- Hardware Multithreading may provide 15—30% speedup with respect to a non-multithreaded processor

A possible realization

- The pipeline stages are separated by two buffers (one for each executing thread)
- If one thread is stalled, the other might go ahead and fill the pipeline slot, improving processor utilization



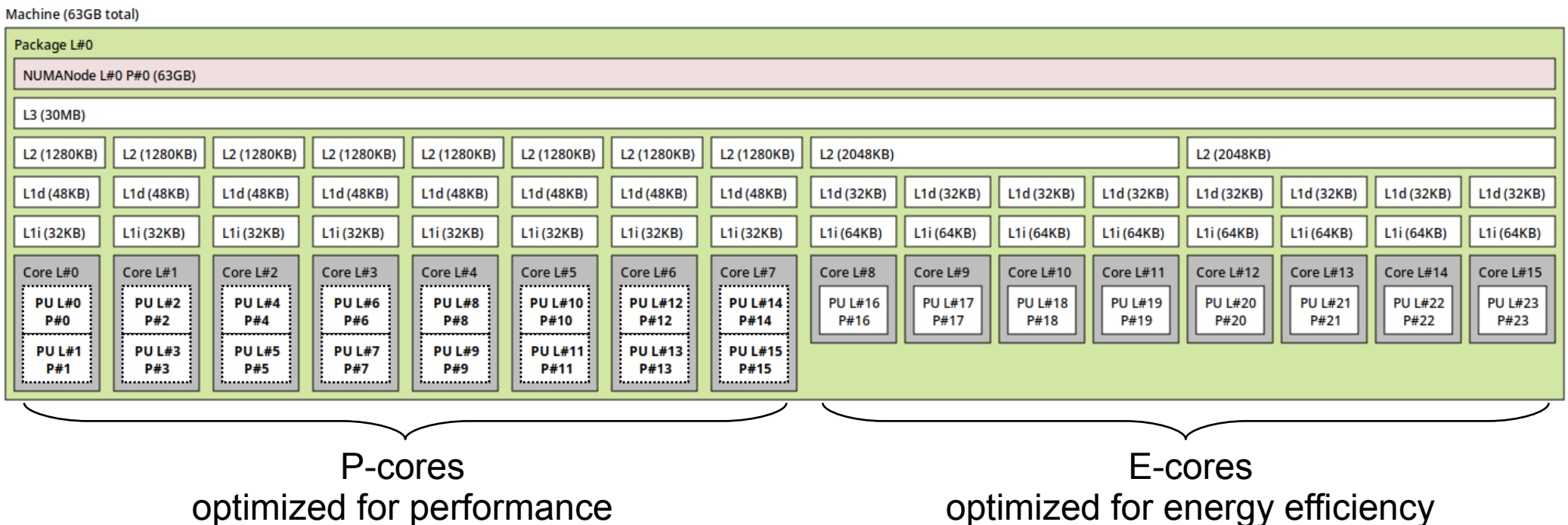
Hyper-Threading Technology Architecture and Microarchitecture

http://www.cs.virginia.edu/~mc2zk/cs451/vol6iss1_art01.pdf

Example

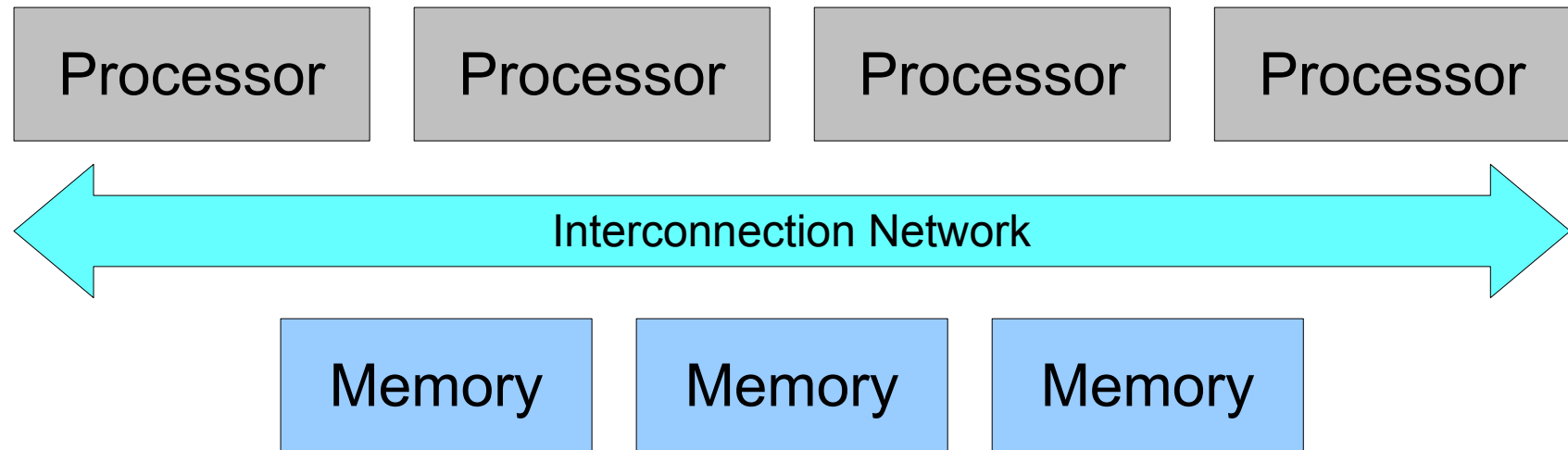
Hyperthreading + Asymmetric cores

- Output of `lscpu` ("Thread(s) per core") or `lstopo` (*hwloc* Ubuntu/Debian package)



Parallel Architectures

An Abstract Parallel Architecture



- How is parallelism handled?
- What is the exact physical location of the memories?
- What is the topology of the interconnect network?

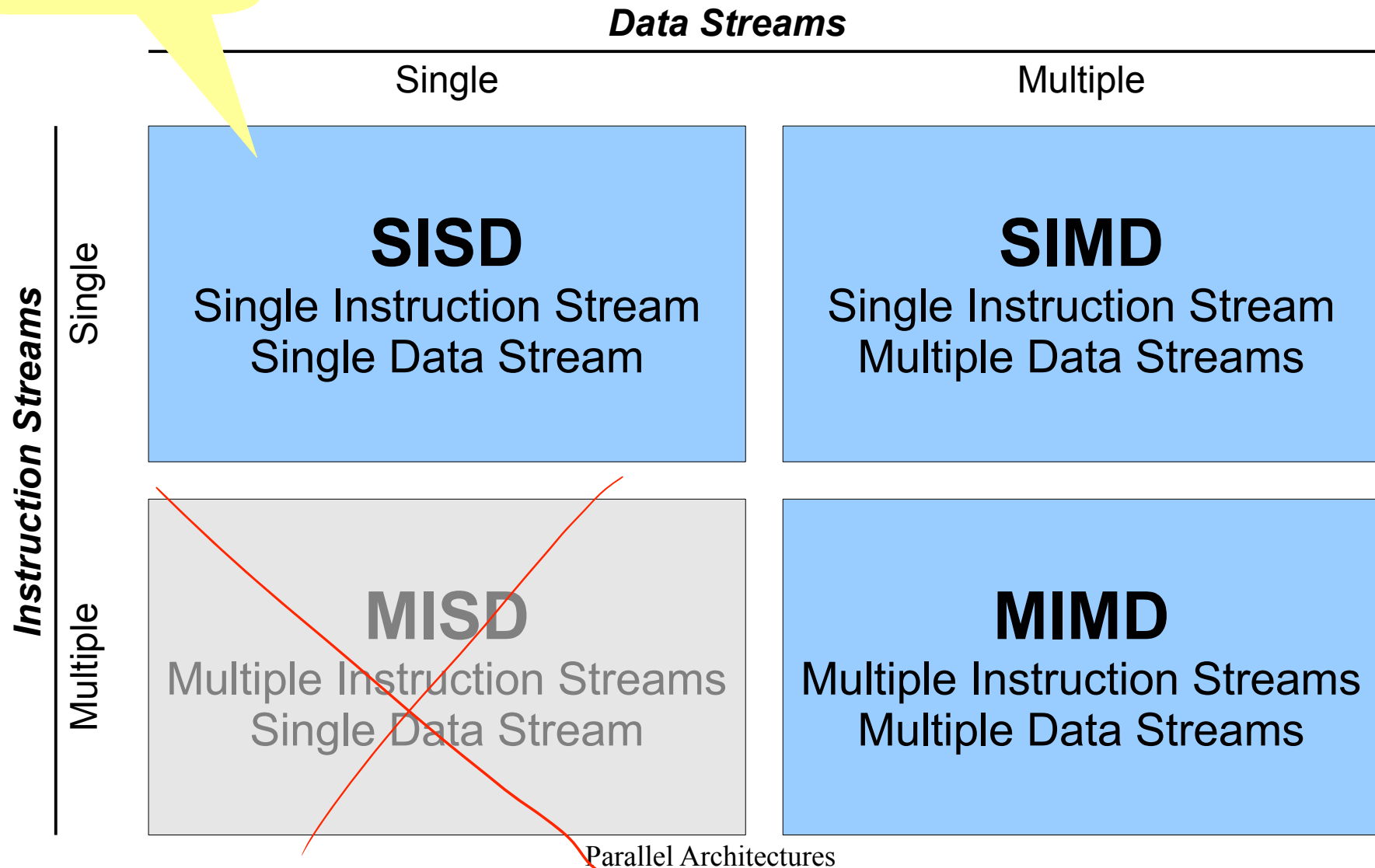
Why are parallel architectures important?

For example there can be multiple memories like in the previous slide or a single one

- There is no "typical" parallel computer: different vendors use different architectures
- There is currently no “universal” programming paradigm that fits all architectures
 - Parallel programs must be tailored to the underlying parallel architecture
 - The architecture of a parallel computer limits the choice of the programming paradigm that can be used

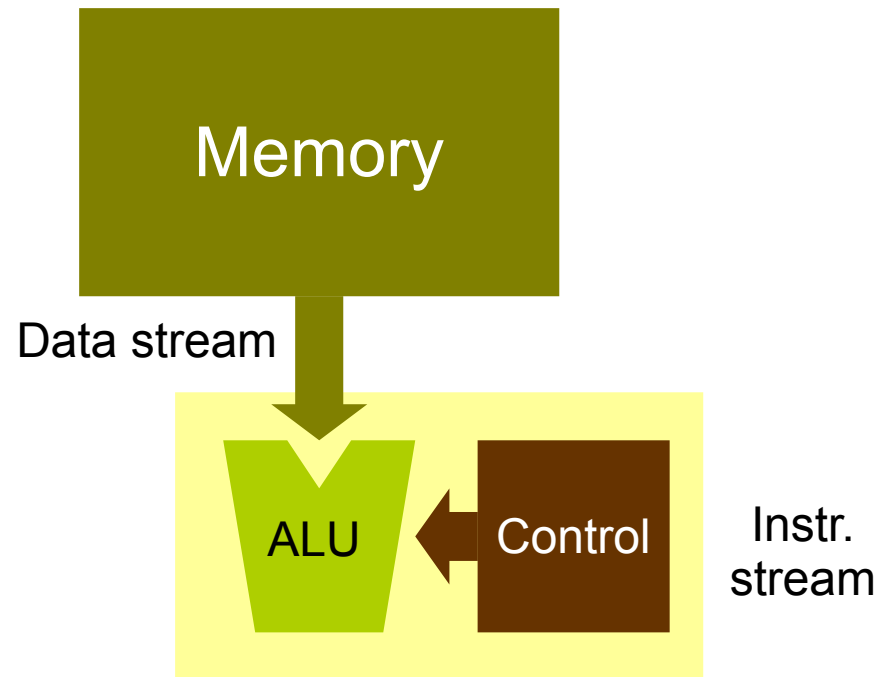
Flynn's Taxonomy

Von Neumann
architecture

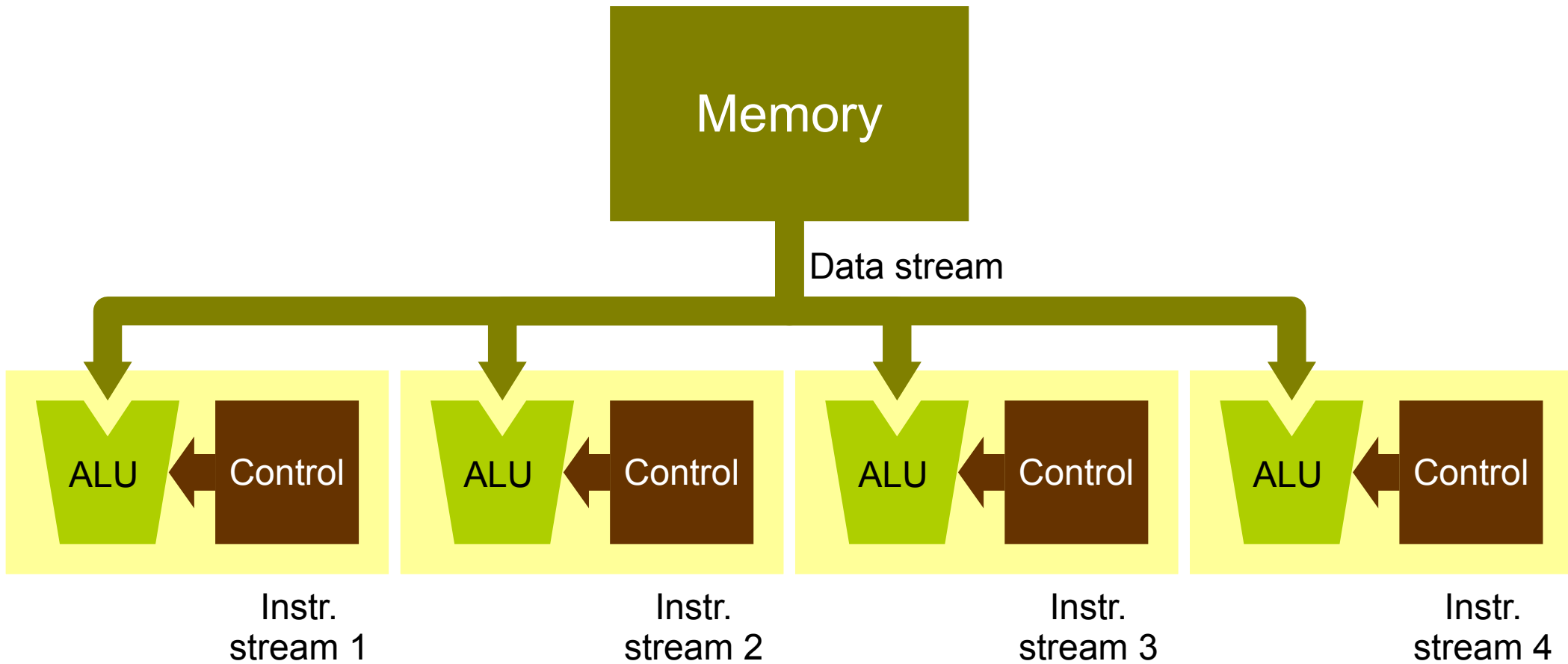


Don't makes sense to force multiple processes on
the single input, they are multiple for a reason ...

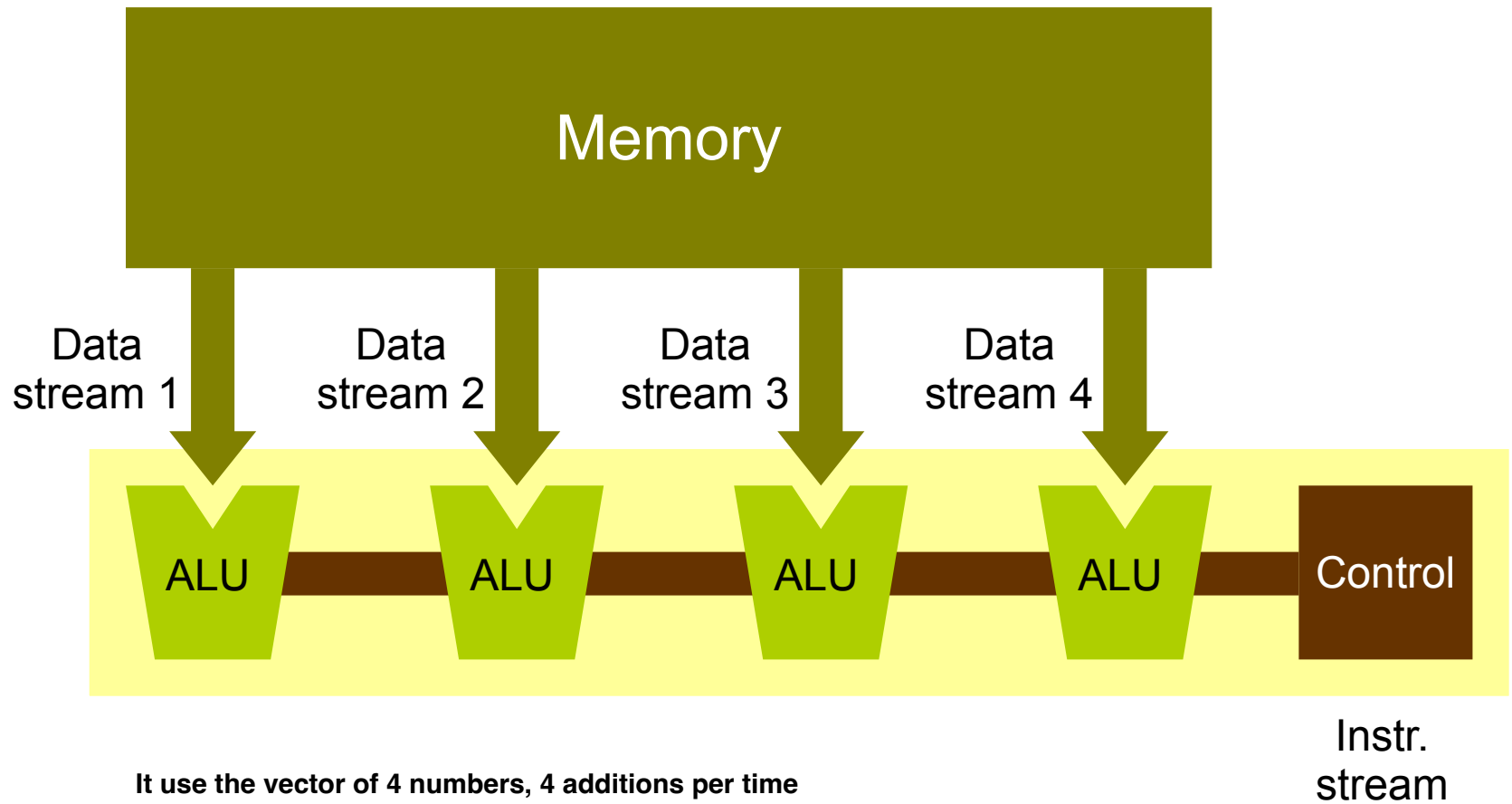
SISD



MISD



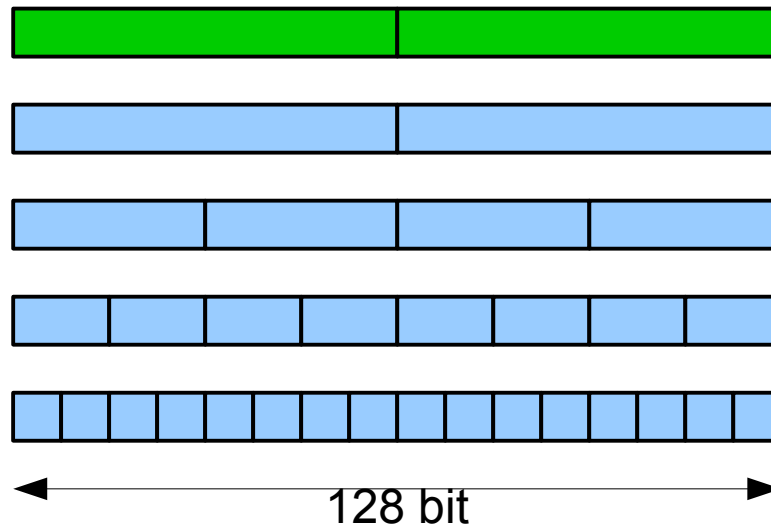
SIMD



SSE

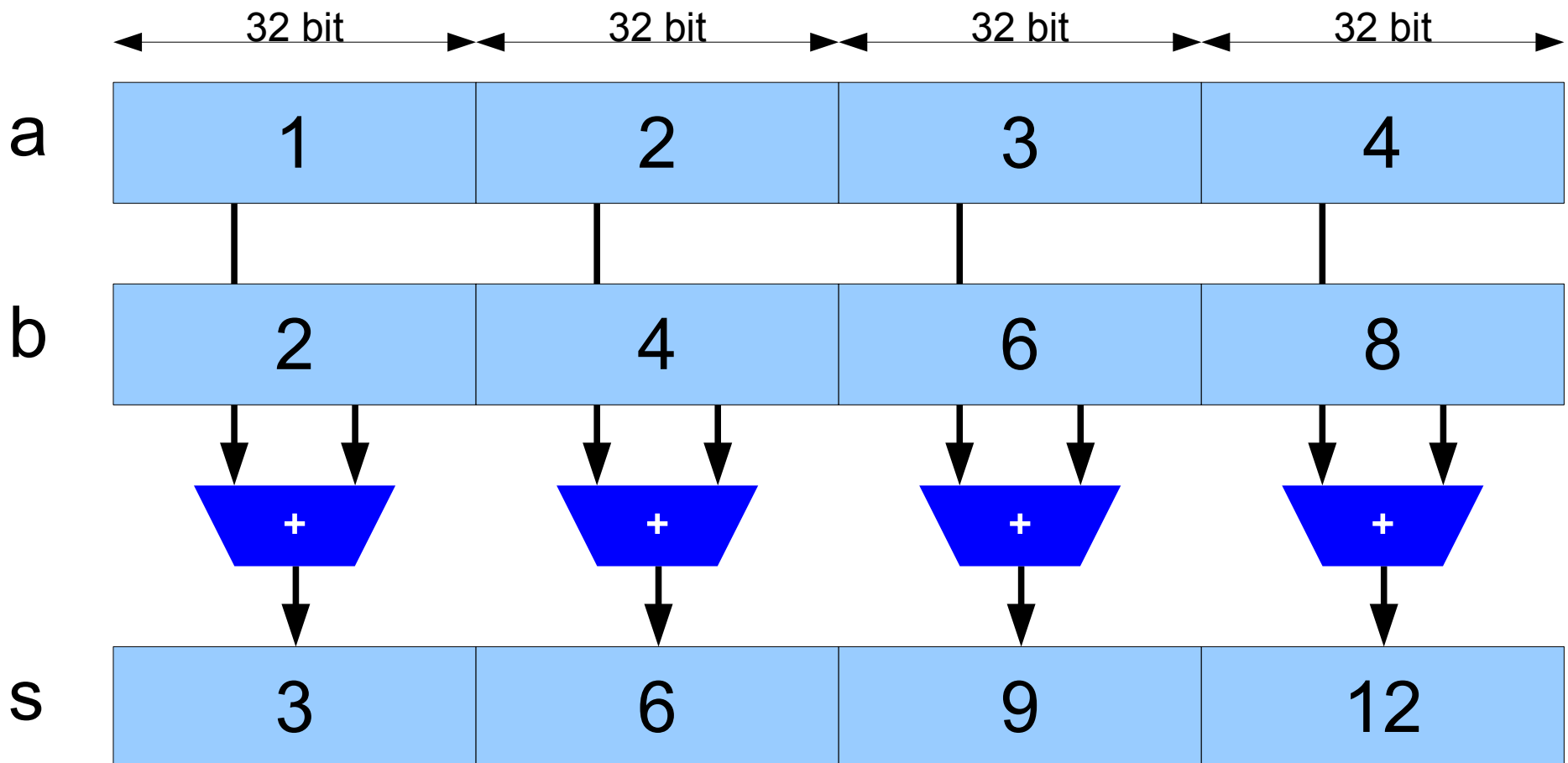
Streaming SIMD Extensions

- Extension to the x86 instruction set
- Provide new SIMD instructions operating on small arrays of integer or floating-point numbers
- Introduces 8 new 128-bit registers (XMM0—XMM7)
- SSE2 instructions can handle
 - 2 64-bit doubles, or
 - 2 64-bit integers, or
 - 4 32-bit integers, or
 - 8 16-bit integers, or
 - 16 8-bit chars

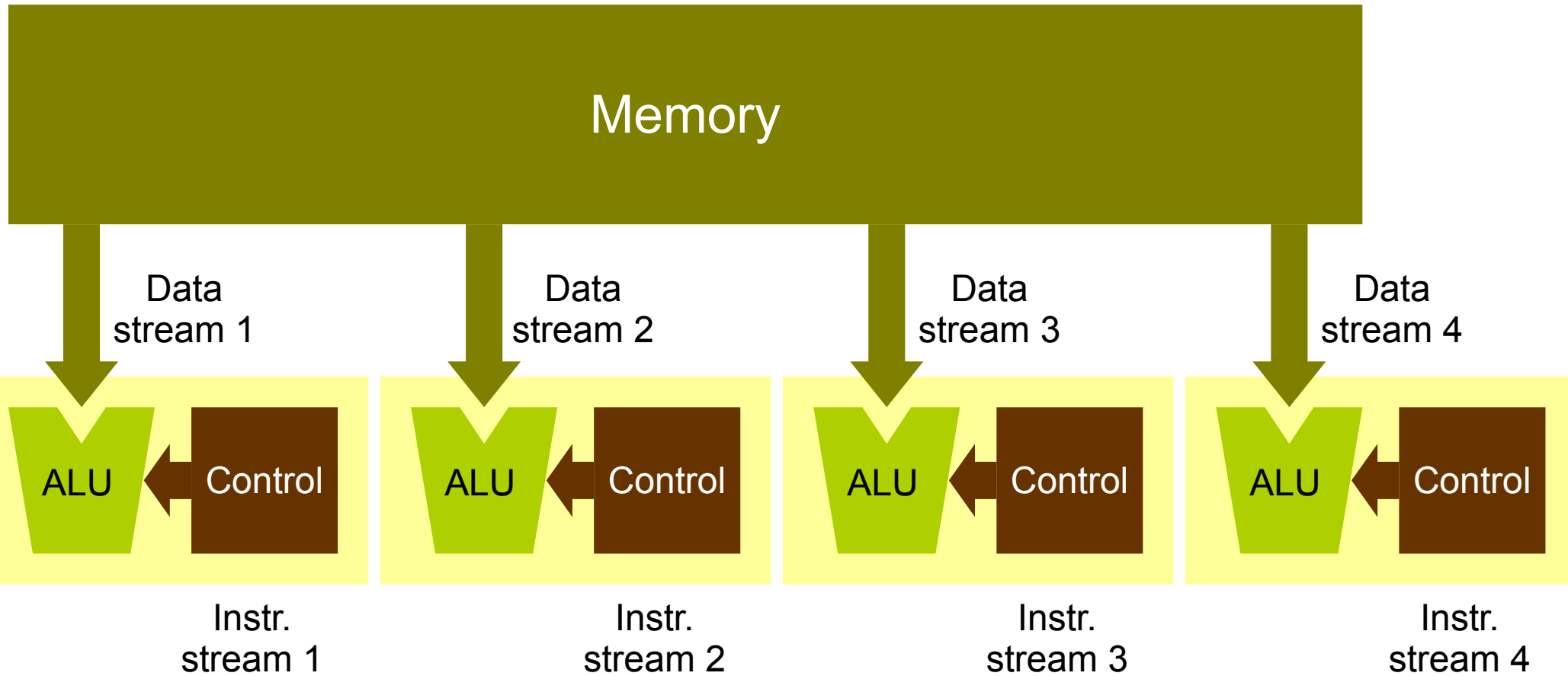


Example

```
__m128i a = _mm_set_epi32( 1, 2, 3, 4 );  
__m128i b = _mm_set_epi32( 2, 4, 6, 8 );  
__m128i s = _mm_add_epi32(a, b);
```



MIMD



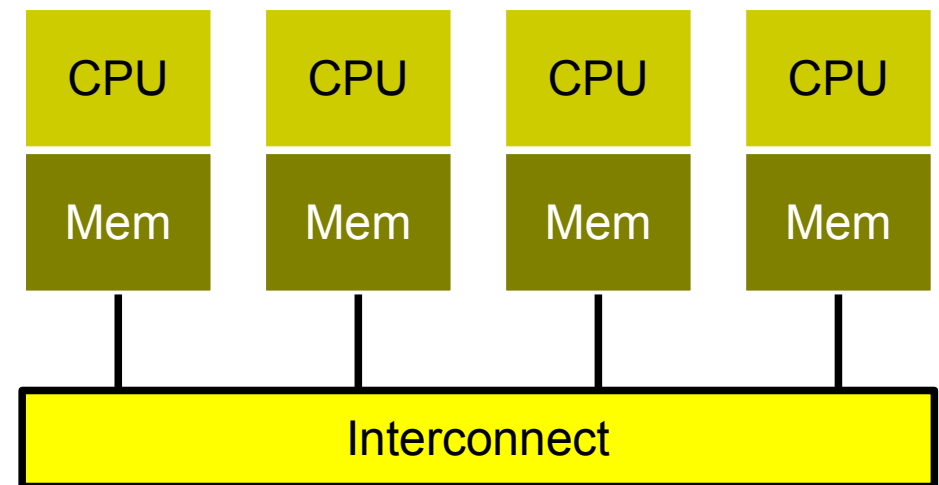
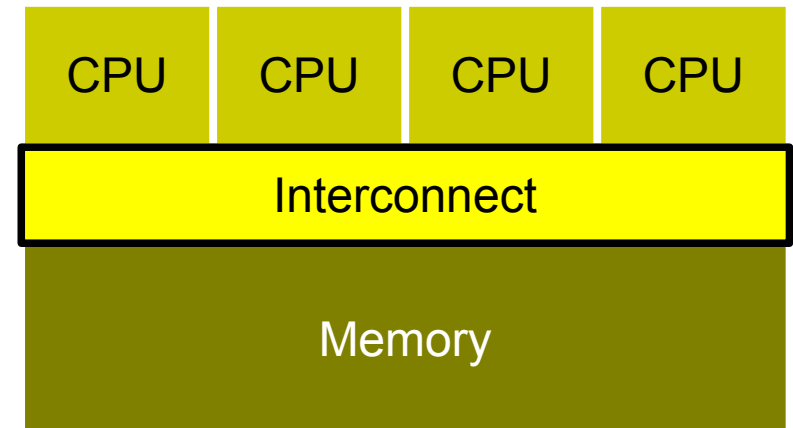
MIMD architectures

- **Shared Memory**

- A set of processors sharing a common memory space
- Each processor can access any memory location

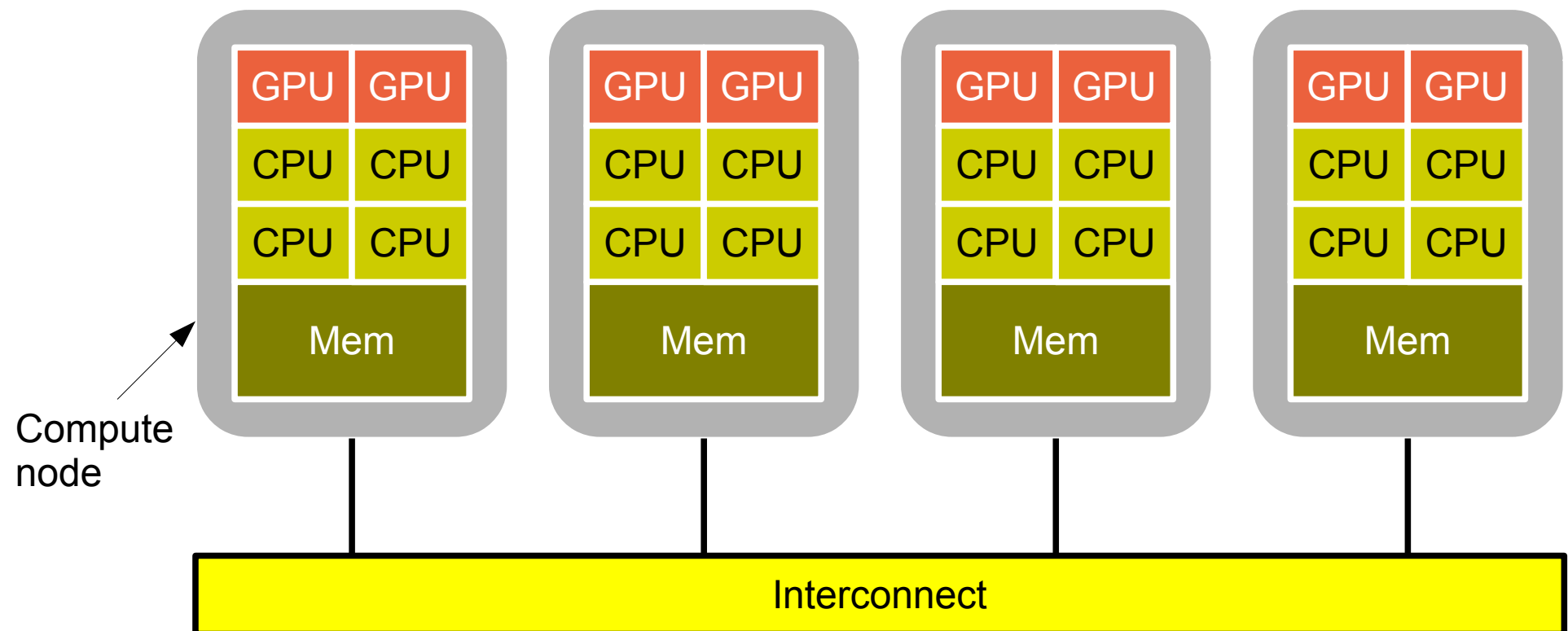
- **Distributed Memory**

- A set of compute nodes connected through an interconnection network
 - The most simple example: cluster of PCs connected via Ethernet
- Nodes can share data through explicit communications



Hybrid architectures

- Many HPC systems are based on hybrid architectures
 - Each compute node is a shared-memory multiprocessor
 - A large number of compute nodes is connected through an interconnect network



www.top500.org

(June 2025)

	Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
	1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
	2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
	3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
	4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
	5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	

www.top500.org

(June 2025)



6

HPC6 - HPE Cray EX235a, AMD Optimized 3rd Generation
EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL
8.9, HPE
Eni S.p.A.
Italy

3,143,520

477.90

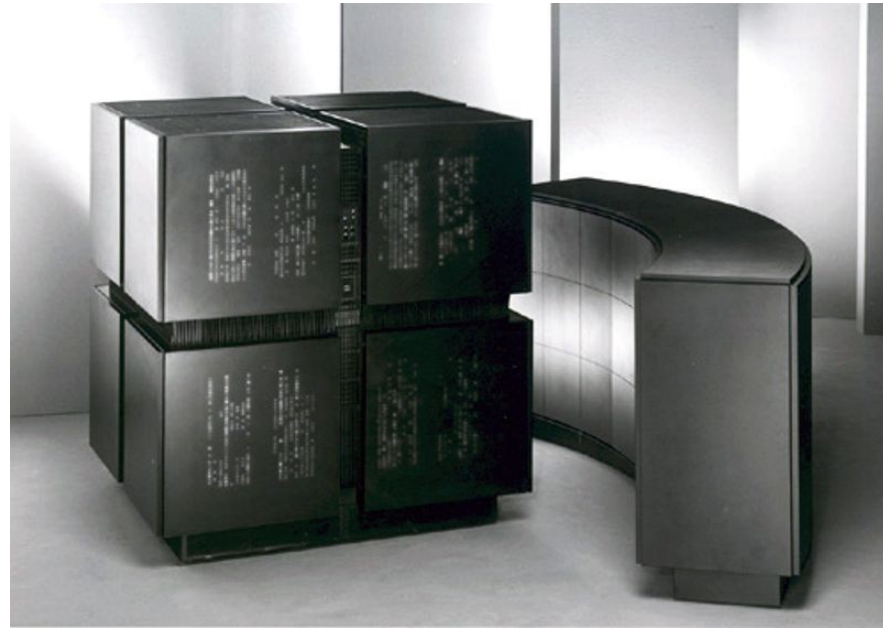
606.97

8,461

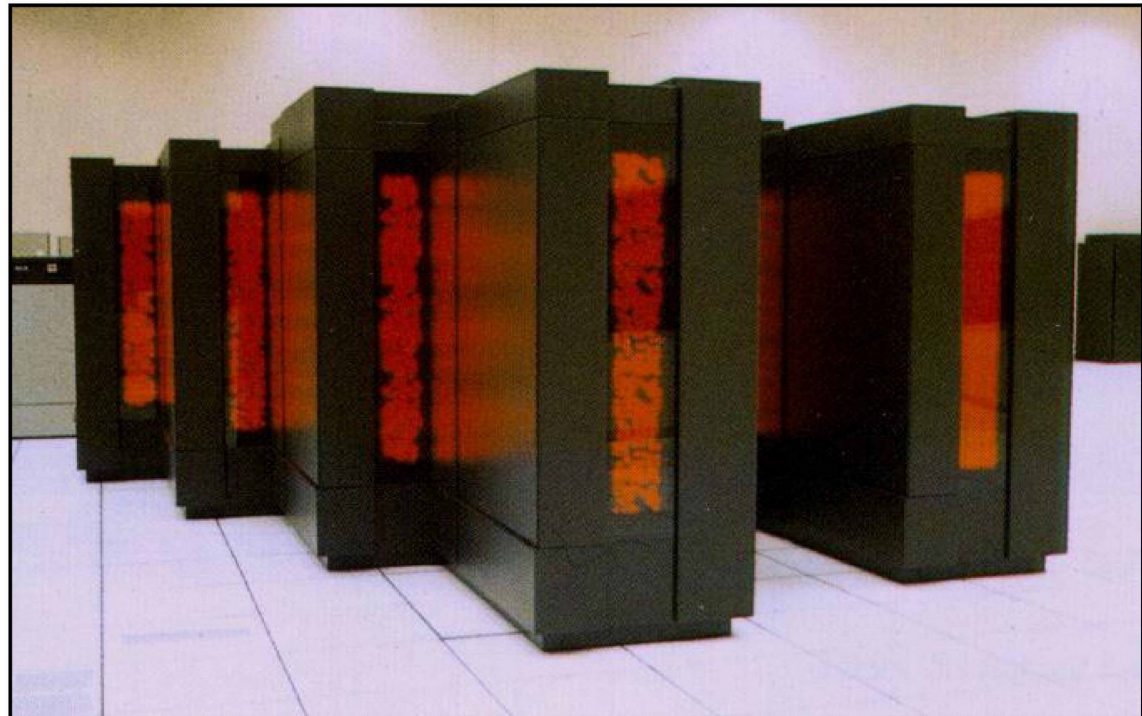


Cray 1 (year 1975)

The C-shape is not only easthetic bit also to minimize the gating



CM-2 (year 1987)



CM-5 (year 1992)



IBM BlueGene/P (year 2007)



Atos BullSequana XH2000 “Leonardo”
(year 2022)

Parallel Architectures

The Computer Programme (BBC), “It's happening now”, 11/1/1982



https://archive.org/details/the_computer_programme_ep01

SANDIA ASCI RED

Date:

1996

Peak performance:

1.8Teraflops

Floor space:

150m²

Power consumption:

800.000 Watt



SANDIA ASCI RED

Date:

1996

Peak performance:

1.8Teraflops

Floor space:

150m²

Power consumption:

800.000 Watt

Sony

PLAYSTATION 3

Date:

2006

Peak performance:

>1.8Teraflops

Floor space:

0.08m²

Power consumption:

<200 Watt



Rules of thumb

- When writing parallel applications (especially on distributed-memory architectures) keep in mind that:
 - Computation is **fast**
 - Communication is **slow**
 - Input/output is **incredibly slow**

Recap

Shared memory

- Advantages:
 - Easier to program
 - Useful for applications with irregular data access patterns (e.g., graph algorithms)
- Disadvantages:
 - The programmer must take care of race conditions
 - Limited memory bandwidth

Distributed memory

- Advantages:
 - Highly scalable, provide very high computational power by adding more nodes
 - Useful for applications with strong locality of reference, with high computation / communication ratio
- Disadvantages:
 - Latency of interconnect network
 - Difficult to program